



МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

**Институт информационных технологий (ИИТ)
Кафедра прикладной математики (ПМ)**

ОТЧЕТ ПО ПРАКТИЧЕСКОЙ РАБОТЕ №1
по дисциплине «Технологии и инструментарий машинного обучения»

Студент группы *ИНБО-20-23. Боргачев Т.М.*

(подпись)

Преподаватель *Трушин СМ*

(подпись)

Москва 2025 г.

СОДЕРЖАНИЕ

1	ПРАКТИЧЕСКАЯ РАБОТА №1.....	3
1.1	Задание 1. Работа с массивами, статистика и обработка данных.	3
1.2	Задание 2. Работа с таблицами в Pandas.....	10
1.3	Задание 3. Визуализация многомерных и категориальных данных.....	17
1.4	Задание 4. Скользящее окно на одномерных данных	25
	ВЫВОД.....	29

1 ПРАКТИЧЕСКАЯ РАБОТА №1

1.1 Задание 1. Работа с массивами, статистика и обработка данных.

Ссылка на Google colab с выполненной работой:
<https://colab.research.google.com/drive/12h195iQYHkYTPpEYy1KnM2RsLRzFfu06?usp=sharing>

Сгенерированный массив с равномерным распределением представлен на Рисунке 1.

```
[-66.4456164 -74.60343028 -85.09008028 -68.06788414 -51.02466239  
-26.13782099 -94.3007286 44.25649993 -50.77918319 -32.08624064  
-54.05146655 -28.39832619 -91.89518903 -50.41688197 49.24600491]
```

Рисунок 1 — Массив с равномерным распределением

Сгенерированный массив с нормальным распределением представлен на Рисунке 2.

```
[12.99010982 21.85995407 12.18400134 11.22591533 8.68007276 10.7834195  
0.03533534 0.76358895 9.13883622 6.22106871 1.51518526 11.98651395  
10.80072565 10.58284152 11.15644043]
```

Рисунок 2 — Массив с нормальным распределением

Сгенерированный массив с экспоненциальным распределением представлен на Рисунке 3.

```
[0.06554198 0.79137429 0.33685053 1.56346319 1.07643656 7.20002358  
2.82476867 0.95709305 4.33651653 0.64699596 0.54619331 0.41104647  
0.17418788 1.81209068 0.02803143]
```

Рисунок 3 — Массив с экспоненциальным распределением

Диаграммы распределений массивов представлены на Рисунке 4.

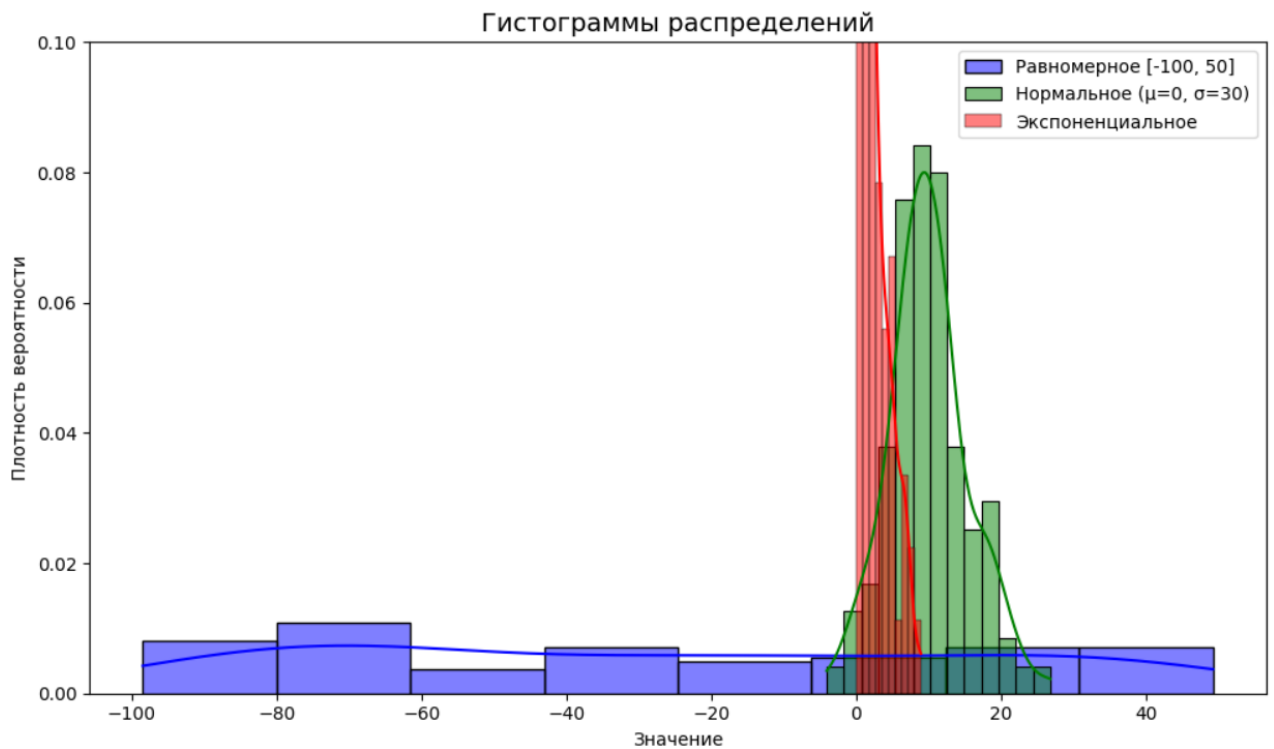


Рисунок 4 — Гистограммы распределений

Ручная нормализация и количество выбросов представлены на Рисунке 5.

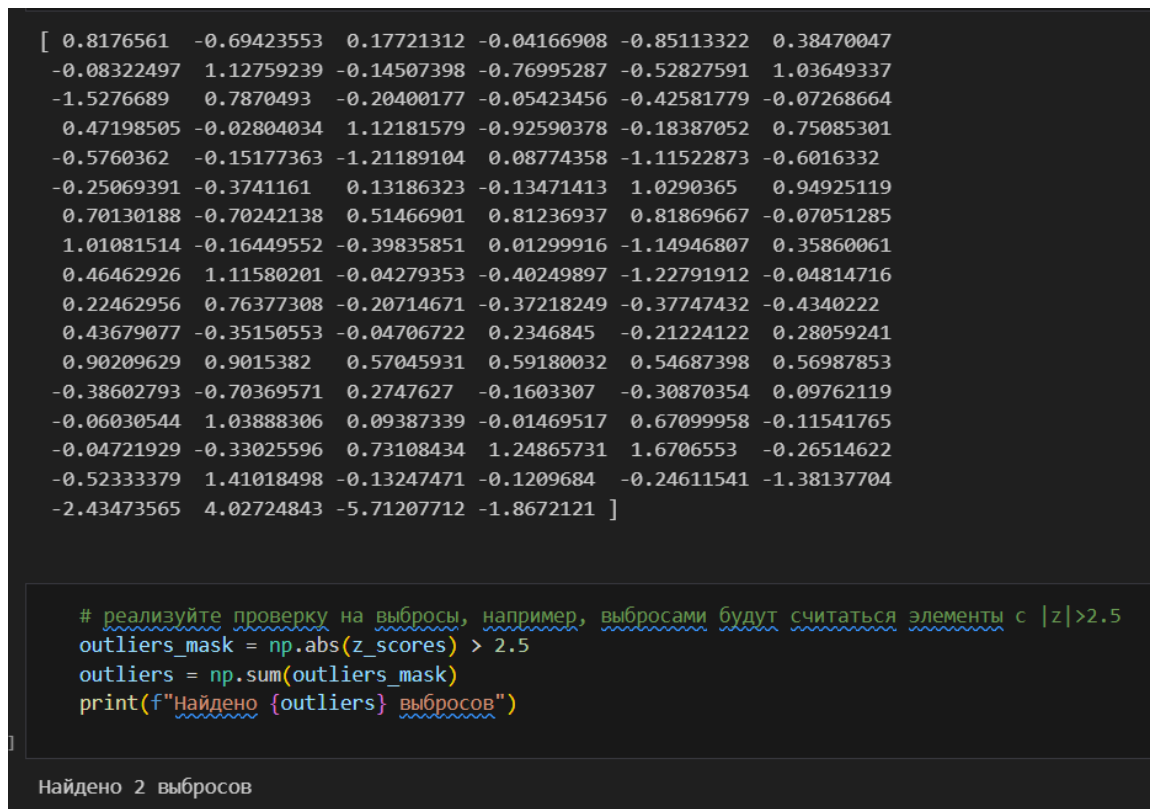


Рисунок 5 — Нормализация Z-score

Визуализация выбросов представлена на Рисунке 6.

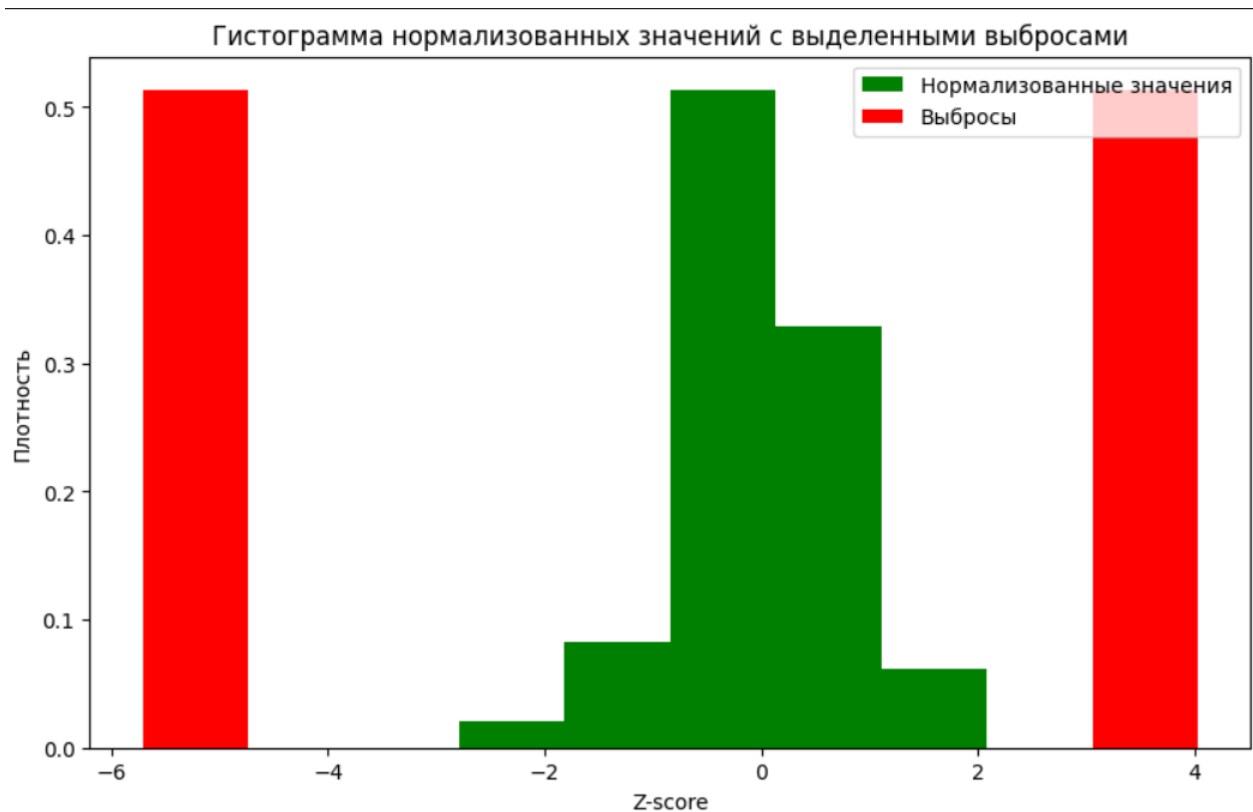


Рисунок 6 — Нормализованные значения и выбросы

Вычисленные статистики представлены на Рисунке 7.

```

Элементов в пределах  $\pm 1\sigma$ : 80
Элементов в пределах  $\pm 1.5\sigma$ : 94

# вычислите вручную ковариацию, дисперсию и корреляцию Пирсона между двумя массивами
# Используем array_uniform и array_normal для примера

# Дисперсия
var_uniform_manual = np.mean((array_uniform - np.mean(array_uniform))**2)
var_normal_manual = np.mean((array_normal - np.mean(array_normal))**2)

# Ковариация
cov_manual = np.mean((array_uniform - np.mean(array_uniform)) * (array_normal - np.mean(array_normal)))

# Корреляция Пирсона
corr_manual = cov_manual / (np.sqrt(var_uniform_manual) * np.sqrt(var_normal_manual))

print(f"Дисперсия array_uniform: {var_uniform_manual}")
print(f"Дисперсия array_normal: {var_normal_manual}")
print(f"Ковариация: {cov_manual}")
print(f"Корреляция Пирсона: {corr_manual}")

Дисперсия array_uniform: 2071.3113449988664
Дисперсия array_normal: 29.173489721551206
Ковариация: 5.284919954949173
Корреляция Пирсона: 0.021499161928833246

```

Рисунок 7 — Рассчитанные статистики

Сравнение производительности выбора подпоследовательностей представлено на Рисунке 8.

```
Цикл: 853626 последовательностей за 9.7828 сек  
NumPy: 853626 последовательностей за 0.1505 сек  
Ускорение: 65.0x
```

Рисунок 8 — Разница между циклом for и numpy

Результат ручного вычисления экспоненциального сглаживания представлен на Рисунке 9.

```
[12.99010982 14.76407867 14.24806321 13.64363363 12.65092146 12.27742107  
 9.82900392 8.01592093 8.24050398 7.83661693 6.5723306 7.65516727  
 8.28427894 8.74399146 9.22648125 10.83156214 11.01051492 10.59889282  
11.5582879 13.04662482 12.62175472 12.30775539 13.53515582 12.34804503  
12.37881528 13.7526432 14.35716029 10.67263363 10.40119343 12.7849553  
11.7624287 10.92855458 10.13422646 13.46968145 14.38163236 13.01070381  
11.12667907 9.77397424 8.16334853 9.07740954 8.30589363 8.93993908  
 9.12473161 9.69784593 8.58457639 8.22894739 7.86394351 8.66507764  
 9.48569074 9.1454152 10.08198881 10.05945092 9.63644193 11.22632085  
11.47456006 10.63756121 10.38899965 10.97521995 10.87615268 8.6674879  
 8.88063201 9.7097444 9.26862874 10.19358462 10.71411529 11.73305303  
11.25417518 10.18122168 10.27506525 10.11076837 9.10745935 9.14610552  
10.64490247 10.2274705 9.75084536 7.64938404 7.27444367 8.05067082  
 7.11491512 6.96277166 7.35150023 6.59529673 7.76530602 8.048914  
10.02275405 9.58100343 11.11416481 12.77981747 11.67290077 10.764162  
10.36805747 9.47776953 8.50588843 10.93960692 10.69906029 8.780965  
 8.02274865 7.74865661 8.05301715 9.0317819 ]
```

Рисунок 9 — Ручное вычисление экспоненциального сглаживания

Результат вычисления экспоненциального сглаживания через библиотеку Pandas представлен на Рисунке 10.

```
[12.99010982 17.91780107 15.56788315 14.09702684 12.48560547 12.02421843
 8.99015196 7.01315416 7.50419726 7.21670198 5.9692427 7.26150021
 8.01052328 8.54865417 9.08923129 10.76904472 10.96491151 10.55502625
11.53735144 13.04728399 12.61832562 12.30268308 13.5383923 12.34499857
12.37649711 13.75495481 14.36047363 10.6681423 10.39718107 12.78470102
11.76121166 10.92691996 10.13241519 13.46992474 14.38219706 13.0107105
11.12619517 9.77330613 8.16254641 9.07688938 8.30539546 8.9395945
 9.12446853 9.69766667 8.5843845 8.22878148 7.86380061 8.66498118
 9.48562822 9.14536033 10.08195561 10.05942415 9.63641743 11.22631054
11.47455297 10.6375524 10.38899186 10.97521513 10.87614863 8.66748127
 8.88062697 9.70974118 9.26862582 10.19358286 10.71411415 11.73305253
11.25417462 10.18122096 10.27506469 10.11076789 9.10745884 9.14610511
10.64490228 10.22747031 9.75084519 7.64938381 7.27444347 8.05067069
 7.11491499 6.96277156 7.35150015 6.59529666 7.76530597 8.04891396
10.02275403 9.58100341 11.1141648 12.77981747 11.67290077 10.76416199
10.36805746 9.47776953 8.50588842 10.93960692 10.69906029 8.780965
 8.02274865 7.74865661 8.05301715 9.0317819 ]
```

Рисунок 10 — Pandas вычисление экспоненциального сглаживания

Визуализация полученных результатов представлена на Рисунке 11.

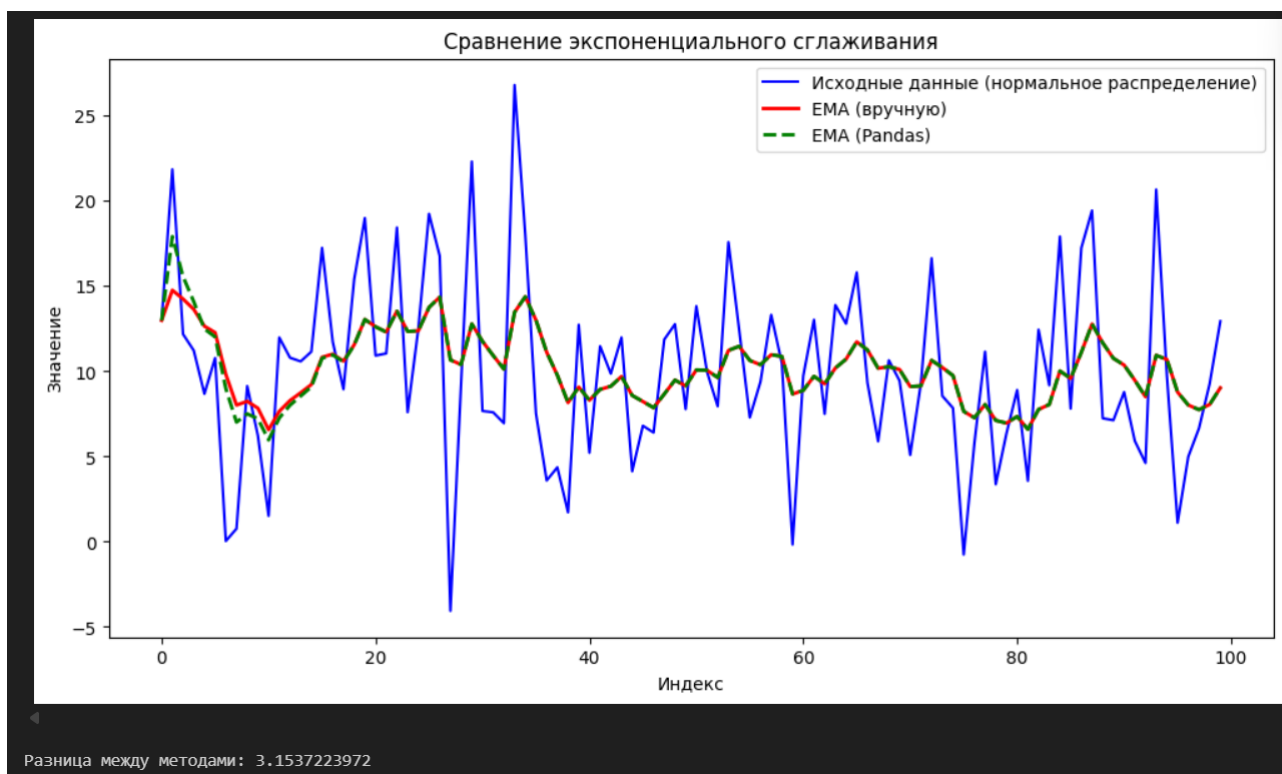


Рисунок 11 — Сравнение экспоненциального сглаживания

Преобразование массива в матрицу представлено на Рисунке 12.

```

Форма матрицы: (10, 10)
[[12.99010982 21.85995407 12.18400134 11.22591533 8.68007276 10.7834195
  0.03533534 0.76358895 9.13883622 6.22106871]
 [ 1.51518526 11.98651395 10.80072565 10.58284152 11.15644043 17.2518857
 11.72632602 8.95240441 15.39586823 18.99997251]
 [10.92227433 11.05175806 18.44475752 7.59960187 12.50189628 19.24795491
 16.77522866 -4.065473 9.31543261 22.32000277]
 [ 7.6723223 7.59305812 6.95691399 26.81150137 18.02943603 7.5269896
 3.59058009 4.36315492 1.72084569 12.73365358]
 [ 5.21982999 11.47612089 9.86390172 11.99030321 4.13149826 6.80643136
 6.403928 11.86961415 12.76814317 7.78431303]
 [13.82828325 9.96929934 7.944406 17.58583651 12.4675169 7.28956579
 9.3947534 13.32010117 10.47988361 -0.16717124]
 [ 9.73320845 13.02619396 7.50416611 13.89340814 12.79623798 15.80880399
 9.33866375 5.88940772 10.65043953 9.45358082]
 [ 5.0942233 9.30069018 16.6400903 8.55774258 7.84434483 -0.75646124
 5.77468217 11.15557945 3.37189228 6.35419786]
 [ 8.90641448 3.57048274 12.44534317 9.18334592 17.91811423 7.81400096
 17.24681032 19.44242814 7.24523398 7.12920688]
 [ 8.78363935 5.91661779 4.61836401 20.67448087 9.7368738 1.10858384
 4.98988324 6.65228846 9.27045928 12.94684093]]

```

Рисунок 12 — Матрица 10x10 из массива

Результат замены локальных минимумов на NaN значения представлен на Рисунке 13.

Локальные минимумы по строкам заменены на NaN

Матрица с NaN

```
[[12.99010982 21.85995407 12.18400134 11.22591533          nan 10.7834195
   nan 0.76358895  9.13883622  6.22106871]
 [ 1.51518526 11.98651395 10.80072565          nan 11.15644043 17.2518857
 11.72632602          nan 15.39586823 18.99997251]
 [10.92227433 11.05175806 18.44475752          nan 12.50189628 19.24795491
 16.77522866          nan  9.31543261 22.32000277]
 [ 7.6723223  7.59305812          nan 26.81150137 18.02943603  7.5269896
   nan 4.36315492          nan 12.73365358]
 [ 5.21982999 11.47612089          nan 11.99030321          nan  6.80643136
   nan 11.86961415 12.76814317  7.78431303]
 [13.82828325  9.96929934          nan 17.58583651 12.4675169          nan
  9.3947534 13.32010117 10.47988361 -0.16717124]
 [ 9.73320845 13.02619396          nan 13.89340814          nan 15.80880399
  9.33866375          nan 10.65043953  9.45358082]
 [ 5.0942233  9.30069018 16.6400903  8.55774258  7.84434483          nan
  5.77468217 11.15557945          nan  6.35419786]
 [ 8.90641448          nan 12.44534317          nan 17.91811423          nan
 17.24681032 19.44242814  7.24523398  7.12920688]
 [ 8.78363935  5.91661779          nan 20.67448087  9.7368738          nan
  4.98988324  6.65228846  9.27045928 12.94684093]]
```

Рисунок 13 — Замена локальных минимумов

Выполненное задание 1 представлено в Листинге 1.1.

Листинг 1.1 — Задание 1

```
#сгенерируйте numpy-массив размером 100 с равномерным
распределением от -100 до 50 (numpy.random.uniform);
import numpy as np

array_uniform = np.random.uniform(-100, 50, 100)
print(array_uniform[:15])

# сгенерируйте массив из 100 случайных значений по
# нормальному распределению со средним 10 и  $\sigma = 5$ 
# (numpy.random.normal);
array_normal = np.random.normal(10, 5, 100)
print(array_normal[:15])
# сгенерируйте массив из 100 значений, распределённых по
# экспоненциальному закону с параметром  $\lambda = 0.5$ 
# (numpy.random.exponential).
array_exponential = np.random.exponential(0.5, 100)
print(array_exponential[:15])
# постройте гистограммы всех трёх массивов на одном графике
```

Продолжение Листинга 1.1

```
# (используйте matplotlib или seaborn), нормируйте плотность
(density=True).
# Подпишите графики: укажите параметры, сравните формы
распределений. Где «тяжёлый хвост»? Где симметрия?
import seaborn as sns
import matplotlib.pyplot as plt
plt.figure(figsize=(10, 6))

sns.histplot(array_uniform, label='Равномерное [-100, 50]',
color='blue', stat='density', kde=True)
sns.histplot(array_normal, label='Нормальное ( $\mu=0$ ,  $\sigma=30$ )',
color='green', stat='density', kde=True)
sns.histplot(array_exponential, label='Экспоненциальное',
color='red', stat='density', kde=True)
plt.ylim(0, 0.1)
plt.title('Гистограммы распределений', fontsize=14)
plt.xlabel('Значение')
plt.ylabel('Плотность вероятности')
plt.legend()
plt.tight_layout()
plt.show()

#У равномерного распределения форма прямоугольная, почти
постоянная плотность
#У нормального распределения наблюдается симметрия относительно
центра. Форма графика – колоколообразная
#У экспоненциального распределения тяжелый хвост. Форма:
спадающая, асимметричная.
```

Таким образом, задание 1 выполнено.

1.2 Задание 2. Работа с таблицами в Pandas

Сгенерированная таблица по заданным параметрам представлена на Рисунке 14.

	user_id	age	income	subscription_date	region	is_active
0	1	56	81.732476	2025-01-10 21:36:20.384016	Запад	True
1	2	46	69.560249	2025-02-28 21:36:20.384016	Запад	True
2	3	32	52.141841	2023-10-26 21:36:20.384016	Запад	True
3	4	60	91.513138	2024-06-23 21:36:20.384016	Север	False
4	5	25	28.013115	2023-09-25 21:36:20.384016	Восток	True

Рисунок 14 — Сгенерированная таблица

Процесс вычисления количества месяцев с момента подписки представлен на Рисунке 15.

```
# вычислите experience_months — количество месяцев с момента подписки
df['experience_months'] = ((pd.Timestamp.now() - df['subscription_date']).dt.days / 30).astype(int)
print(df['experience_months'])
```

✓ 0.0s

```
0      8
1      6
2     22
3     14
4     23
..
495    14
496     6
497    13
498    24
499    11
Name: experience_months, Length: 500, dtype: int64
```

Рисунок 15 — Результат вычисления experience_months

Процесс вычисления квантилей дохода представлен на Рисунке 16.

```
# создайте категориальный признак "income_group" по квантилям дохода
df['income_group'] = pd.qcut(df['income'], q=4, labels=['Q1', 'Q2', 'Q3', 'Q4'])
print(df['income_group'])
```

✓ 0.0s

0	Q4
1	Q3
2	Q2
3	Q4
4	Q1
	..
495	Q2
496	Q3
497	Q1
498	Q1
499	Q4

Name: income_group, Length: 500, dtype: category
Categories (4, object): ['Q1' < 'Q2' < 'Q3' < 'Q4']

Рисунок 16 — Результат вычисления income_group

Процесс вычисления потребительской оценки представлен на Рисунке 17.

```
# постройте новую метрику – loyalty_score = income / (1 + experience_months)
df['loyalty_score'] = df['income'] / (1 + df['experience_months'])
print(df['loyalty_score'])
```

✓ 0.0s

0	9.081386
1	9.937178
2	2.267037
3	6.100876
4	1.167213
	...
495	4.109861
496	9.864758
497	2.091328
498	1.051327
499	7.690165

Name: loyalty_score, Length: 500, dtype: float64

Рисунок 17 — Результат вычисления loyalty_score

Поиск среднего дохода активных пользователей по регионам представлен на Рисунке 18.

```
# найдите средний доход активных пользователей по регионам
avg_income_by_region = df[df['is_active'] == True].groupby('region')['income'].mean()
print("Средний доход активных пользователей по регионам:")
print(avg_income_by_region)
```

✓ 0.0s

Средний доход активных пользователей по регионам:

region	
Восток	69.917265
Запад	63.960071
Север	62.583037
Центр	61.680837
Юг	61.034761

Name: income, dtype: float64

Рисунок 18 — Результат вычисления среднего дохода

Сводная таблица среднего `loyalty_score` по регионам и группам дохода представлена на Рисунке 19.

Сводная таблица среднего `loyalty_score` по регионам и группам дохода:

income_group	Q1	Q2	Q3	Q4
region				
Восток	4.085425	9.603175	24.978809	7.405031
Запад	5.927313	7.515662	16.591228	11.478510
Север	5.474256	10.140783	7.286187	19.067026
Центр	3.592925	7.595971	11.928946	8.915772
Юг	6.094829	10.479838	9.791537	11.856264

Рисунок 19 — Построенная сводная таблица

Результат поиска самых активных покупателей представлен на Рисунке 20.

Найдено 25 пользователей (5% от 500) с наибольшим loyalty_score				
	user_id	loyalty_score	income	region
120	121	117.211033	117.211033	Север
381	382	87.267418	87.267418	Юг
349	350	83.449108	83.449108	Север
350	351	80.178193	80.178193	Запад
452	453	77.259417	77.259417	Восток
249	250	75.376258	75.376258	Восток
459	460	75.057483	75.057483	Восток
84	85	68.430329	68.430329	Восток
254	255	68.123088	68.123088	Восток
5	6	66.732110	66.732110	Запад
261	262	66.669875	66.669875	Восток
341	342	66.146554	66.146554	Запад
42	43	65.886223	65.886223	Запад
407	408	65.756761	65.756761	Восток
445	446	64.215499	64.215499	Центр
215	216	61.304635	61.304635	Восток
294	295	60.990899	60.990899	Север
464	465	55.571625	55.571625	Юг
16	17	53.646550	53.646550	Запад
101	102	42.352619	42.352619	Юг
264	265	41.310380	41.310380	Запад
262	263	40.698385	81.396770	Юг
92	93	38.701763	38.701763	Север
193	194	38.421179	76.842357	Запад
329	330	36.676488	73.352976	Запад

Рисунок 20 — 5% пользователей с наивысшим loyalty_score

Замена пропусков средним значением представлена на Рисунке 21.

```
# случайно удалите 5% значений из income, замените их средним значением по региону
df_with_missing = df.copy()

# Определяем индексы для удаления (5% от данных)
n_missing = int(len(df_with_missing) * 0.05)
missing_indices = np.random.choice(df_with_missing.index, size=n_missing, replace=False)

# Устанавливаем NaN
df_with_missing.loc[missing_indices, 'income'] = np.nan

# Заменяем пропущенные значения средним по региону
df_with_missing['income'] = df_with_missing.groupby('region')['income'].transform(
    lambda x: x.fillna(x.mean())
)

print(f"Удалено {n_missing} значений income и заменено средним по региону")
✓ 0.0s
```

Удалено 25 значений income и заменено средним по региону

Рисунок 21 — Результат замены пропусков

Процесс интерполяции представлен на Рисунке 22.

```
# проведем интерполяцию по времени подписки (если есть пропуски)

df_sorted = df_with_missing.sort_values('subscription_date').copy()
df_for_interpolation = df_sorted.copy()
interpolation_indices = np.random.choice(df_for_interpolation.index[:100], size=5, replace=False)
df_for_interpolation.loc[interpolation_indices, 'income'] = np.nan
df_for_interpolation = df_for_interpolation.set_index('subscription_date').sort_index()
df_for_interpolation['income_interpolated'] = df_for_interpolation['income'].interpolate(method='time')
df_for_interpolation.head()
```

✓ 0.0s

	user_id	age	income	region	is_active	experience_months	income_group	loyalty_score	income_interpolated
subscription_date									
2023-09-11 21:36:20.384016	499	22	26.283172	Запад	True	24	Q1	1.051327	26.283172
2023-09-11 21:36:20.384016	384	26	33.243057	Запад	False	24	Q1	1.329722	33.243057
2023-09-12 21:36:20.384016	133	54	67.359715	Восток	True	24	Q4	3.289620	67.359715
2023-09-12 21:36:20.384016	75	48	76.729790	Центр	True	24	Q3	3.069192	76.729790
2023-09-16 21:36:20.384016	201	58	77.598132	Восток	False	24	Q3	3.103925	77.598132

Рисунок 22 — Интерполяция по времени подписки

Функция контроля значений представлена на Рисунке 23.

```

# реализуйте функцию контроля значений: выбрасывайте строки, где income или age стали некорректными (например, отрицательными)
def validate_data(dataframe):
    initial_count = len(dataframe)

    # Удаляем строки с отрицательным income или age
    valid_df = dataframe[(dataframe['income'] >= 0) & (dataframe['age'] >= 0)]

    final_count = len(valid_df)
    removed_count = initial_count - final_count

    print(f"Изначально строк: {initial_count}")
    print(f"Удалено некорректных строк: {removed_count}")
    print(f"Осталось строк: {final_count}")

    return valid_df

# Применяем функцию к датафрейму
df_cleaned = validate_data(df_with_missing)
✓ 0.0s

Изначально строк: 500
Удалено некорректных строк: 0
Осталось строк: 500

```

Рисунок 23 – Функция контроля значений

Выполненное задание 2 представлено в Листинге 1.2.

Листинг 1.2 — Задание 2

```

# реализуйте нормализацию z-score вручную, без использования
scikit-learn и stats.zscore

# Основные данные (95% значений)
main_data = np.random.normal(0, 1, 95)

# Выбросы (5% значений из распределения с большим разбросом)
outlier_data = np.random.normal(0, 5, 5)

# Объединяем
mixed_data = np.concatenate([main_data, outlier_data])

mean_val = np.mean(mixed_data)
std_val = np.std(mixed_data)
z_scores = (mixed_data - mean_val) / std_val
print(z_scores)

# реализуйте проверку на выбросы, например, выбросами будут
считаться элементы с |z|>2.5
outliers_mask = np.abs(z_scores) > 2.5
outliers = outliers_mask.sum()
print(f"Найдено {outliers} выбросов")

# постройте гистограмму нормализованных значений и выделите
выбросы другим цветом
import matplotlib.pyplot as plt
plt.figure(figsize=(10, 6))
plt.hist(z_scores, color='green', label='Нормализованные
значения', density=True)
plt.hist(z_scores[outliers_mask], color='red',
label='Выбросы', density=True)
plt.xlabel('Z-score')

```

Продолжение Листинга 1.2

```
plt.ylabel('Плотность')
plt.title('Гистограмма нормализованных значений с выделенными
выбросами')
plt.legend()
plt.show()
# отфильтруйте все элементы, находящиеся в пределах  $\pm 1\sigma$ , затем
– в пределах  $\pm 1,5\sigma$ . Сравните количество таких элементов
within_1_sigma = np.abs(z_scores) <= 1
within_1_5_sigma = np.abs(z_scores) <= 1.5

count_1_sigma = np.sum(within_1_sigma)
count_1_5_sigma = np.sum(within_1_5_sigma)

print(f"Элементов в пределах  $\pm 1\sigma$ : {count_1_sigma}")
print(f"Элементов в пределах  $\pm 1.5\sigma$ : {count_1_5_sigma}")
# вычислите вручную ковариацию, дисперсию и корреляцию Пирсона
между двумя массивами
# Используем array_uniform и array_normal для примера

# Дисперсия
var_uniform_manual = np.mean((array_uniform -
np.mean(array_uniform))**2)
var_normal_manual = np.mean((array_normal -
np.mean(array_normal))**2)

# Ковариация
cov_manual = np.mean((array_uniform - np.mean(array_uniform))
* (array_normal - np.mean(array_normal)))

# Корреляция Пирсона
corr_manual = cov_manual / (np.sqrt(var_uniform_manual) *
np.sqrt(var_normal_manual))

print(f"Дисперсия array_uniform: {var_uniform_manual}")
print(f"Дисперсия array_normal: {var_normal_manual}")
print(f"Ковариация: {cov_manual}")
print(f"Корреляция Пирсона: {corr_manual}")
```

Таким образом, задание 2 – выполнено.

1.3 Задание 3. Визуализация многомерных и категориальных данных

Описание и первые строки датасета “diamonds” представлены на Рисунке

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 53940 entries, 0 to 53939
Data columns (total 10 columns):
#   Column      Non-Null Count  Dtype
---  -
0   carat       53940 non-null  float64
1   cut         53940 non-null  category
2   color       53940 non-null  category
3   clarity     53940 non-null  category
4   depth       53940 non-null  float64
5   table       53940 non-null  float64
6   price       53940 non-null  int64
7   x           53940 non-null  float64
8   y           53940 non-null  float64
9   z           53940 non-null  float64
dtypes: category(3), float64(6), int64(1)
memory usage: 3.0 MB
None

```

	carat	cut	color	clarity	depth	table	price	x	y	z
0	0.23	Ideal	E	SI2	61.5	55.0	326	3.95	3.98	2.43
1	0.21	Premium	E	SI1	59.8	61.0	326	3.89	3.84	2.31
2	0.23	Good	E	VS1	56.9	65.0	327	4.05	4.07	2.31
3	0.29	Premium	I	VS2	62.4	58.0	334	4.20	4.23	2.63
4	0.31	Good	J	SI2	63.3	58.0	335	4.34	4.35	2.75

Рисунок 24 — Датасет “diamonds”

Результат удаления дубликатов и поиска пропусков представлен на Рисунке 25.

```

Количество дубликатов: 146
Количество пропусков по столбцам:
carat      0
cut        0
color      0
clarity    0
depth      0
table      0
price      0
x          0
y          0
z          0
dtype: int64
Дубликаты удалены. Новый размер: (53794, 10)

```

Рисунок 25 — Дубликаты и пропуски

Диаграмма распределения цены бриллиантов представлена на Рисунке 26.

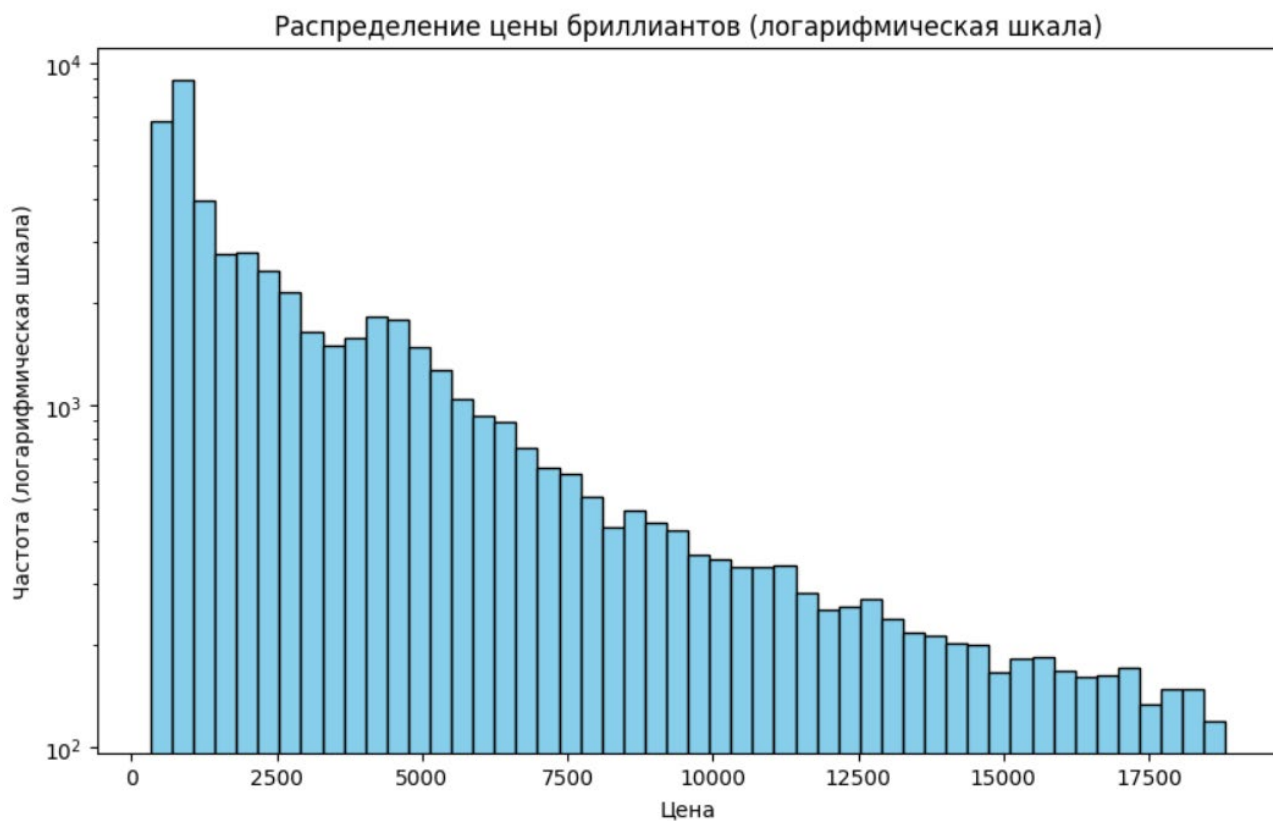


Рисунок 26 — Распределение цены бриллиантов

Построенные boxplot'ы по цене представлены на Рисунке 27.

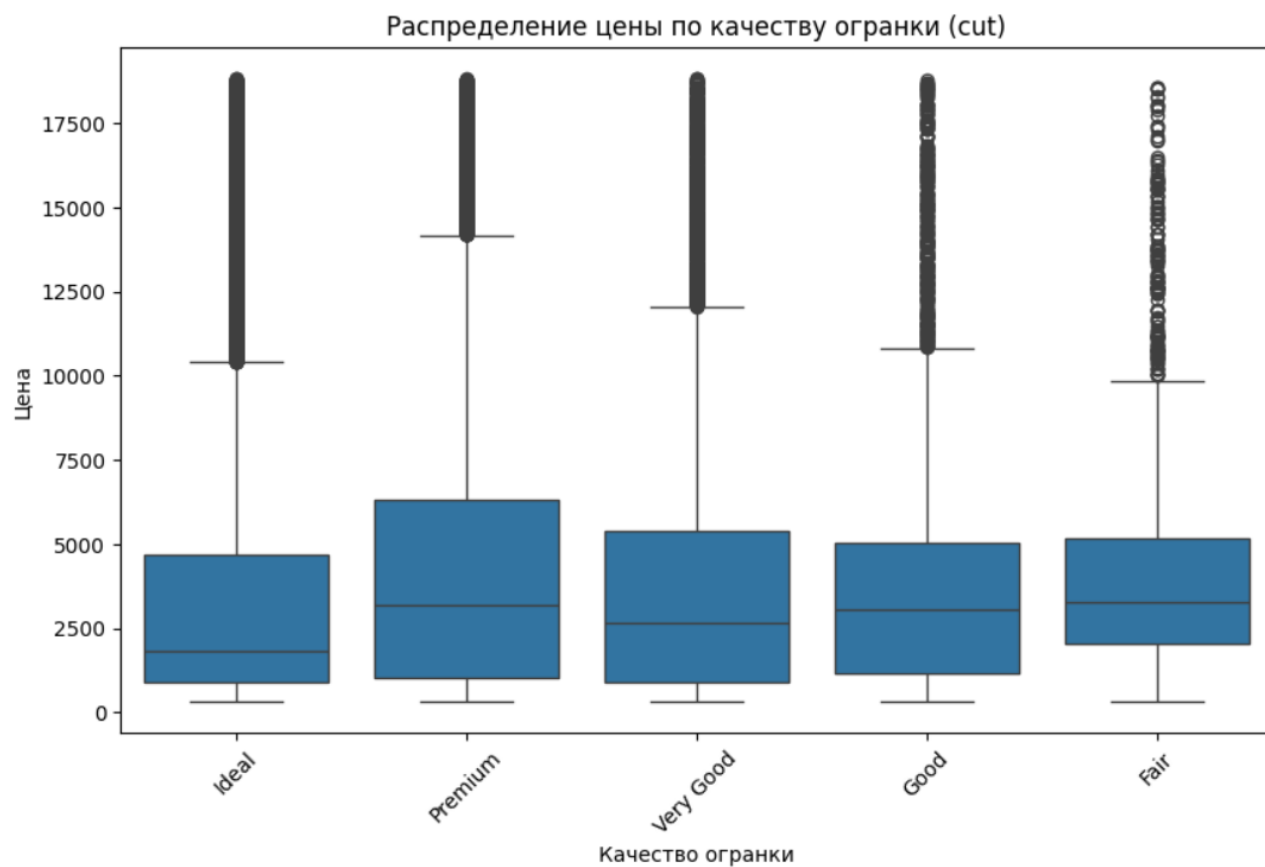


Рисунок 27 — Распределение цены по качеству огранки

На диаграммах видно, что большая часть цен сконцентрирована в низких значениях, но чем качественнее огранка, тем больше высоких значений.

Построенные violinplot'ы представлены на Рисунке 28.

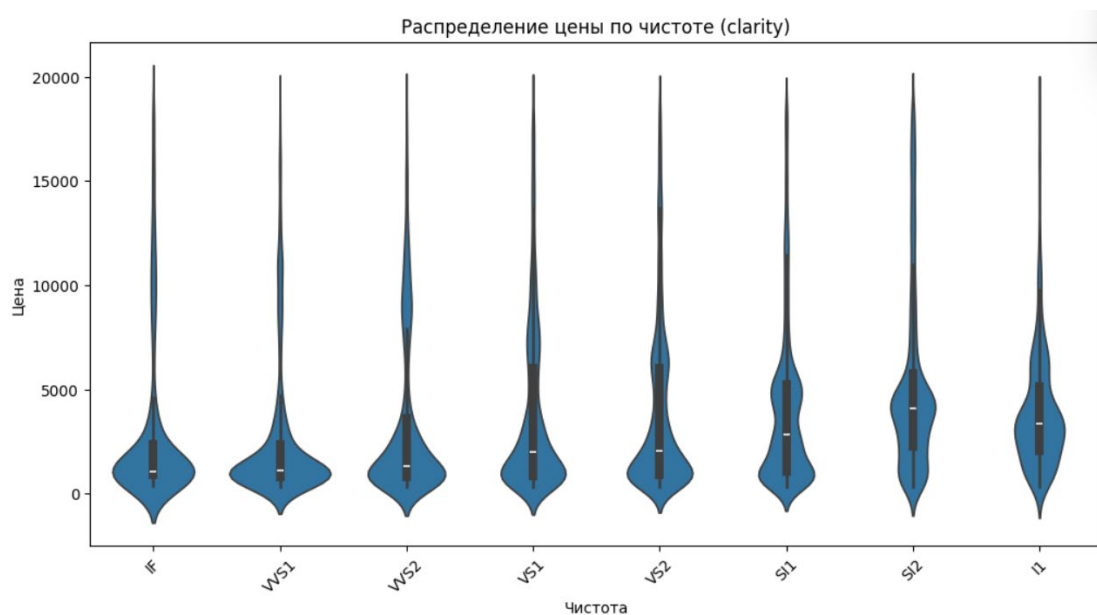


Рисунок 28 — Распределение цен по чистоте

Наилучшее распределение наблюдается у I1, но, как и в прошлом случае,

данные сконцентрированы в районе цен от 0 до 5000.

Тепловая карта корреляции числовых признаков представлена на Рисунке 29.



Рисунок 29 — Тепловая карта корреляции

На графике видно, что цена сильно коррелирует с размером бриллианта. А также длина, ширина и глубина коррелируют между друг другом и размером.

Построенный pairplot представлен на Рисунке 30.

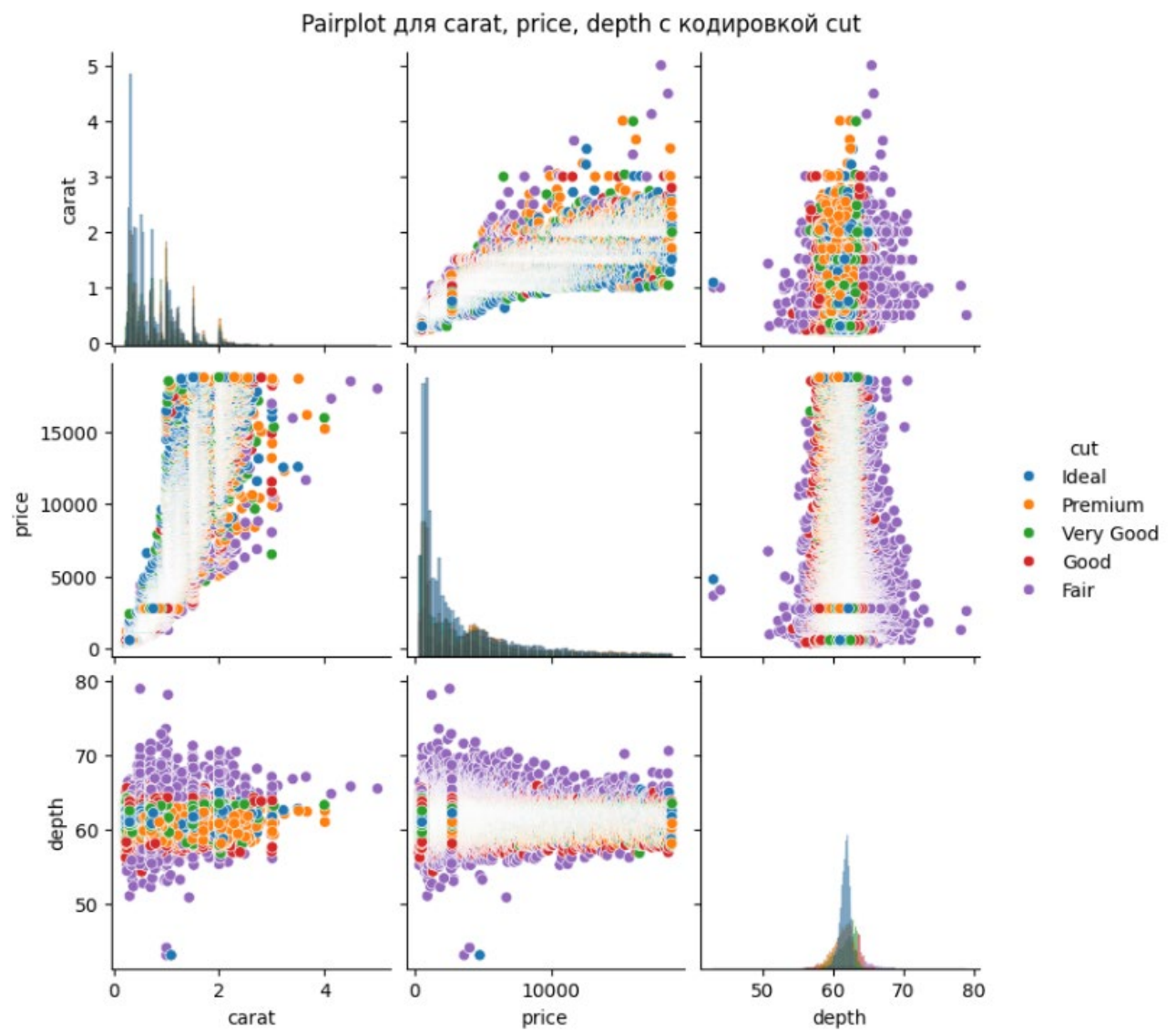


Рисунок 30 — Построенный pairplot

Построенная сводная таблица средней цены представлена на Рисунке 31.

Сводная таблица средней цены по cut и color:						
color \ cut	D	E	F	G	H	I
Ideal	2635.239107	2601.217056	3378.721131	3728.445838	3896.114369	
Premium	3634.493433	3542.385242	4327.297286	4498.782728	5217.907274	
Very Good	3472.662252	3215.471030	3777.724457	3874.210183	4524.835074	
Good	3408.107576	3425.748657	3497.254405	4114.074799	4246.585122	
Fair	4291.061350	3689.743243	3834.750809	4227.000000	5091.307692	

color \ cut	I	J
Ideal	4452.125359	4922.871365
Premium	5934.466245	6287.733251
Very Good	5257.477140	5110.304284
Good	5032.471042	4587.833333
Fair	4607.534483	4975.655462

Рисунок 31 — Сводная таблица средней цены по cut и color

Тепловая карта средней цены представлена на Рисунке 32.



Рисунок 32 — Тепловая карта средней цены бриллиантов

Данные о самых дорогих бриллиантах представлены на Рисунке 33.

Наиболее дорогие бриллианты:

Качество огранки: Premium

Цвет: J

Средняя цена: 6288

Закономерности:

1. Более высокое качество огранки (Ideal) обычно соответствует более высокой цене
2. Цвета (J, I, H) обычно дороже

Рисунок 33 — Информация о дорогих бриллиантах

Закономерности:

1. Чем выше качество огранки (Ideal), тем выше цена.
2. Цвета (J, I, H) обычно дороже.

Выполненное задание 3 представлено в Листинге 1.3.

Листинг 1.3 — Задание 3

```
# выделите из равномерного массива все подпоследовательности
длинной 3, в которых сумма > 30: через цикл for
import time
array_uniform = np.random.uniform(-100, 50, 100000)

start_time = time.time()
subsequences_loop = []
for i in range(len(array_uniform) - 2):
    subseq = array_uniform[i:i+3]
    if np.sum(subseq) > 30:
        subsequences_loop.append(subseq)
time_loop = time.time() - start_time

print(f"Найдено {len(subsequences_loop)}
подпоследовательностей через цикл for")

print(f"Время выполнения (цикл): {time_loop:.6f} секунд")
# выделите из равномерного массива все подпоследовательности
длинной 3, в которых сумма > 30: с помощью NumPy
start_time = time.time()

# Создаем скользящие окна
from numpy.lib.stride_tricks import sliding_window_view
windows = sliding_window_view(array_uniform, window_shape=3)
# Считаем суммы
sums = np.sum(windows, axis=1)

# Находим индексы, где сумма > 30
mask = sums > 30
subsequences_numpy = windows[mask]

time_numpy = time.time() - start_time
```

Продолжение Листинга 1.3

```
print(f"Найдено {len(subsequences_numpy)}  
подпоследовательностей через NumPy")  
print(f"Время выполнения (NumPy): {time_numpy:.6f} секунд")  
print(f"NumPy быстрее в {time_loop/time_numpy:.2f} раз" if  
time_numpy > 0 else "NumPy быстрее")  
# реализуйте вручную алгоритм вычисления экспоненциального  
сглаживания (EMA) с фактором затухания 0.2 для массива с нормальным  
распределением  
alpha = 0.2  
ema_manual = np.zeros_like(array_normal)  
ema_manual[0] = array_normal[0]  
for i in range(1, len(array_normal)):  
    ema_manual[i] = alpha * array_normal[i] + (1 - alpha) *  
ema_manual[i-1]  
  
print(ema_manual)  
# реализуйте алгоритм вычисления экспоненциального  
сглаживания (EMA) с фактором затухания 0.2 через библиотеку Pandas  
import pandas as pd  
  
series = pd.Series(array_normal)  
ema_pandas = series.ewm(alpha=0.2).mean().values  
  
print(ema_pandas)  
# визуализируйте полученные EMA и сравните результаты  
plt.figure(figsize=(12, 6))  
plt.plot(array_normal, label='Исходные данные (нормальное  
распределение)', color='blue')  
plt.plot(ema_manual, label='EMA (вручную)', linewidth=2,  
color='red')  
plt.plot(ema_pandas, label='EMA (Pandas)', linewidth=2,  
linestyle='--', color='green')  
plt.xlabel('Индекс')  
plt.ylabel('Значение')  
plt.title('Сравнение экспоненциального сглаживания')  
plt.legend()  
plt.show()  
  
print(f"Разница между методами: {np.max(np.abs(ema_manual -  
ema_pandas)):.10f}")
```

Таким образом, задание 3 – выполнено.

1.4 Задание 4. Скользящее окно на одномерных данных

Функция скользящего окна и её тестирование представлены на Рисунке 34.

```

# Необходимо реализовать функцию скользящего окна для заданного одномерного ряда данных
def sliding_window(x, w, step=1):
    if len(x) < w:
        return np.array([]).reshape(0, w)
    n_windows = (len(x) - w) // step + 1
    result = np.zeros((n_windows, w))
    for i in range(n_windows):
        start_idx = i * step
        end_idx = start_idx + w
        if end_idx <= len(x):
            result[i] = x[start_idx:end_idx]
        else: break
    return result

# Проверить корректность работы программы с помощью следующего кода
window, step_s = 3, 1
x1 = np.array([8, 1, 4, 5, -2, 5, 9, 0])
A1 = np.array([[8, 1, 4],
               [1, 4, 5],
               [4, 5, -2],
               [5, -2, 5],
               [-2, 5, 9],
               [5, 9, 0]])
result1 = sliding_window(x1, w=window, step=step_s)
print(np.array_equal(result1, A1))
window, step_s = 2, 4
x2 = np.array([8, 3, 4, 1, -6, 5, 9, 2, 10, 11, -14, 0])
A2 = np.array([[8, 3],
               [-6, 5],
               [10, 11]])
result2 = sliding_window(x2, w=window, step=step_s)
print(np.array_equal(result2, A2))
✓ 0.0s
True
True

```

Рисунок 34 — Скользящее окно

Выполненное задание 4 представлено в Листинге 1.4.

Листинг 1.4 — Задание 4

```

# преобразуйте массив в матрицу 10×10 (reshape)
matrix_10x10 = array_normal.reshape(10, 10)
print(f"Форма матрицы: {matrix_10x10.shape}")
print(matrix_10x10)

# установите в NaN (np.nan) те элементы, которые являются
локальными минимумами по строкам
matrix_with_nan = matrix_10x10.copy()

for i in range(matrix_with_nan.shape[0]):
    row = matrix_with_nan[i]

```

Продолжение Листинга 1.4

```
        for j in range(1, len(row)-1):
            if row[j] < row[j-1] and row[j] < row[j+1]:
                matrix_with_nan[i, j] = np.nan

    print("Локальные минимумы по строкам заменены на NaN")
    print("Матрица с NaN")
    print(matrix_with_nan)
    # Необходимо реализовать функцию скользящего окна для
    заданного одномерного ряда данных
    def sliding_window(x, w, step=1):
        """
        Создает матрицу скользящих окон из одномерного массива

        Параметры:
        x : array-like, одномерный массив
        w : int, размер окна
        step : int, шаг между окнами (по умолчанию 1)

        Возвращает:
        numpy.ndarray, матрица скользящих окон
        """
        if len(x) < w:
            return np.array([]).reshape(0, w)

        # Количество окон
        n_windows = (len(x) - w) // step + 1

        # Создаем матрицу для результатов
        result = np.zeros((n_windows, w))

        # Заполняем матрицу
        for i in range(n_windows):
            start_idx = i * step
            end_idx = start_idx + w
            if end_idx <= len(x):
                result[i] = x[start_idx:end_idx]
            else:
                break

        return result

    print("Функция скользящего окна реализована")
    # Проверить корректность работы программы с помощью следующего
кода
    window = 3
    step_s = 1
    x1 = np.array([8, 1, 4, 5, -2, 5, 9, 0])
    A1 = np.array([[8, 1, 4],
                    [1, 4, 5],
                    [4, 5, -2],
```

Продолжение Листинга 1.4

```
        [5, -2, 5],
        [-2, 5, 9],
        [5, 9, 0]])
result1 = sliding_window(x1, w=window, step=step_s)
print("Тест 1:")
print(f"Результат совпадает: {np.array_equal(result1, A1)}")
print("Ожидаемый результат:")
print(A1)
print("Полученный результат:")
print(result1)
window = 2
step_s = 4
x2 = np.array([8, 3, 4, 1, -6, 5, 9, 2, 10, 11, -14, 0])
A2 = np.array([[8, 3],
               [-6, 5],
               [10, 11]])
result2 = sliding_window(x2, w=window, step=step_s)
print("\nТест 2:")
print(f"Результат совпадает: {np.array_equal(result2, A2)}")
print("Ожидаемый результат:")
print(A2)
print("Полученный результат:")
print(result2)
```

Таким образом, задание 4 – выполнено.

ВЫВОД

В ходе выполнения практической работы №1 были успешно освоены ключевые методы генерации, обработки и анализа данных с использованием инструментов Python, таких как NumPy, Pandas и библиотек визуализации. На первом этапе исследованы основные законы распределений: равномерное, нормальное и экспоненциальное. Гистограммы наглядно продемонстрировали их отличительные особенности — симметричность нормального распределения, равномерность плотности для равномерного закона и наличие «тяжёлого хвоста» у экспоненциального, что подчеркнуло важность выбора подходящей модели в зависимости от характера данных. Реализация z-нормализации и выявления выбросов вручную позволила глубже понять принципы статистического анализа, а сравнение количества элементов в пределах $\pm 1\sigma$ и $\pm 1,5\sigma$ подтвердило теоретические ожидания, связанные с правилом трёх сигм. Вычисление ковариации и корреляции как вручную, так и через библиотечные функции показало их согласованность, что укрепило доверие к используемым методам.

В работе с табличными данными сгенерированный DataFrame позволил отработать навыки создания производных признаков, таких как стаж подписки и `loyalty_score`, а также агрегации данных по регионам и группам дохода. Обработка пропусков через замену средним по группе и интерполяцию по времени продемонстрировала гибкость Pandas в решении задач очистки данных. Анализ датасета `diamonds` выявил сильную корреляцию между весом бриллианта (`carat`) и его ценой, а визуализация через тепловые карты и `violinplot` позволила обнаружить закономерности: например, бриллианты с огранкой `Ideal` и цветом `D` оказались наиболее дорогими, что соответствует логике рынка. Особое внимание уделено интерпретации выбросов в зависимости от чистоты камня, что важно для задач оценки качества.

Реализация скользящего окна и экспоненциального сглаживания подчеркнула разницу между ручными алгоритмами и готовыми решениями

библиотек, а их визуальное сравнение показало высокую точность вычислений. Работа с масками и индексацией в NumPy, включая обработку локальных минимумов, расширила понимание эффективных методов фильтрации данных.